RoboCup Rescue 2019 - TDP Agent Simulation SOS-CS (Iran)

Armin Kazemi, Hamid Ziari, Ava Abderezaei, Zahra Golpayegani, Mona Mehrabizadeh Honarmand, Saeed Shiry Ghidary
arminkz@live.com aut.ziari@gmail.com

March 4, 2019

Abstract This paper is a summary of SOS-CS team efforts on rescue agent simulation league in RoboCup 2019 Competitions. We introduce new agent strategies and base code improvements we developed this year, such as a new search algorithm for ambulance agents, improvements in path planning module, and etc. According to our experiments, better results are achieved due to these improvements. Also, main modules and agent strategies are briefly explained.

Keywords: RoboCup, Rescue Simulation, search, path planning

1 Introduction

The SOS-CS basic operations is same as previous years as described in our previous TDP, so in this paper we mainly focus on new modules added to our code. next the changes that comes to agents strategy will be discussed to benefit higher scores.

2 Modules

2.1 Path Planning

Purpose Basically purpose of Path Planning is to plan movements of agents and describe it in a step by step manner, so it could avoid blocked roads and also chose the best path to reach the target.

Related Works There are several algorithms to use for path planning such as Floyd-Warshall, A*, dijkstra, and etc. each algorithm has its pros and cons. for example Floyd-Warshall algorithm is a all-source all-destination algorithm and eliminates the need for recalculating after changing the source location, but due its high complexity most teams avoid using it and go for using A* and Dijkstra algorithms which are single-source algorithms. for determining reachability within roads there is also a lot of methods. for example Grid-BFS method (used by team Aura) and a method which is based on ratio of blockades area to road area (which is used by team MRL) we used a whole different method which we will discuss later.

Proposed Approach The first thing we needed for developing path planning was to create a world graph of the whole map $G(V, E)$. To do so we assumed each passable edge of area as a node (V) of graph (G). nodes are connected by an edge (E) if there is a path from one node of area to another node.

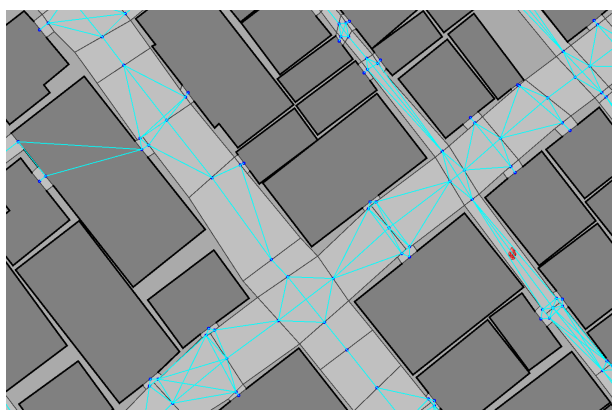
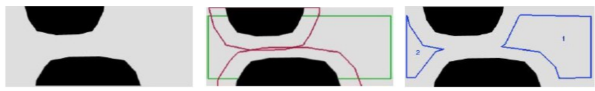


Figure 1: World Graph

In order to determine reachability between nodes we used a technique same as in real life robots. In fact the algorithm expands each blockade of

area by half of the agent's radius and then subtracts that from the original area. In the resulting polygon(s) if two nodes were both in the same polygon, then two nodes are reachable to each other.



(a) A (b) B (c) C

Figure 2: Node to Node Reachability

for finding the best path whithin our world graph, we've used a customized version of Dijkstra's algorithm. We don't use Floyd-Warshall algorithm because it has a rather high time complexity. A* algorithm is a single target algorithm which is not an optimum choice due to the need of calculating all targets and giving scores to them in one pass. We have implemented dijkstra algorithm using Fibonacci Heap data structure instead of using a queue to save space and time.

Pros and Cons The pros of this approach is that Due to its all-destination nature, we can calculate best route to all possible targets in just one pass. The cons of this approach comparing to Floyd-Warshall is whenever the starting point changes, calculations should be done again.

2.2 Clustering

Purpose Clustering purpose is to limit the scope of operation for agents and there for paying attention to all regions of map equally. another benefit of clustering is to prevent log distance movements for agents because it mainly keeps the agent in its own cluster area and prevents long travels for agent task.

Related Works There are so many approaches to handle clustering by participating teams. but more practical ones are KMeans algorithm and DBSCAN algorithm.but because clustering problem in rescue maps are usually linearly partitive we see that most teams prefer to use KMeans algorithm for clustering.

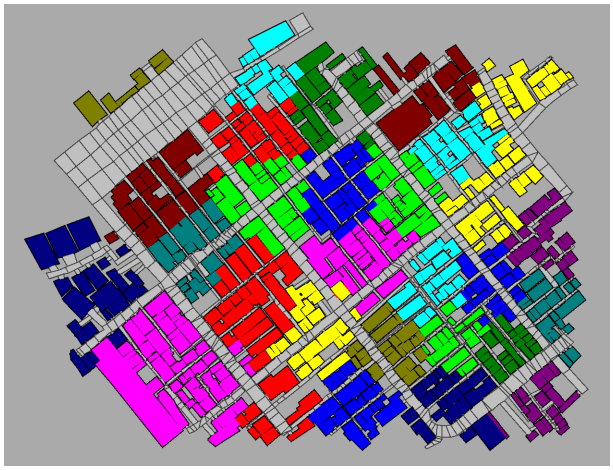
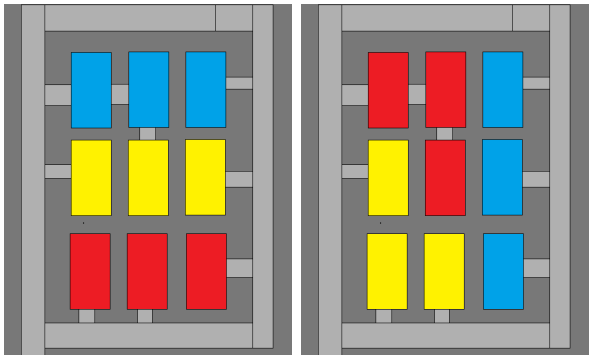


Figure 3: Clusters in 'Kobe' Map using KMeans

Proposed Approach Suppose buildings presented as below. KMeans clustering is applied to these buildings, but this clustering is not an optimum choice. For example, suppose an agents wants to approach the yellow cluster. This agent needs to travel a long distance to get there. But our customized KMeans clustering does so by calculating routes to the buildings. This method helps the agent travel less in order to see a buildings, so it is a more optimum way to achieve clustering. Our idea is to calculate the distance between all buildings of the map using an algorithm such as a customized version of Bellman-Ford algorithm, then do clustering using our customized KMeans to minimize sum of distances inside clusters. The center of a cluster is a building that has the least sum of distances among other buildings of that cluster.



(a) Clustering using normal KMeans (b) Clustering using customized KMeans

Agents have one main cluster and one or more backup clusters. Priority for each agent is to act on its own main cluster. If there is no job available in the main cluster agent resorts to backup clusters. assignment is done by Hungarian algorithm with use of heuristic function of distance between agent and cluster. Number of main clusters is equal to number of agents as implied before.

Pros and Cons The pros of this method are:
1- Travel cost would become less, so this method saves us time.
2- This method can improve performance of agents especially in ambulance team agents.

The cons and challenges of this method are:
1- Computation and space complexity of this method is higher than normal KMeans.
2- There is a possibility that our customized KMeans doesn't converge.

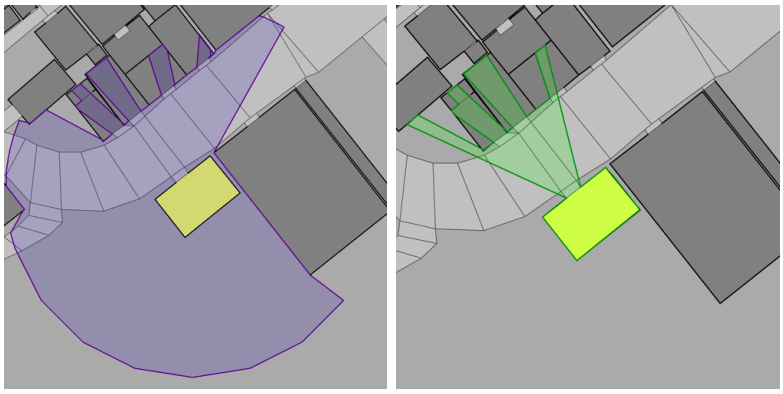
We hope to find a solution to the cons of this method by the start of competitions.

2.3 Determining the Perception Area

In order to prevent unnecessary movements and save agent the crucial time, we must calculate the area which each building could be percepted within. To do so we take advantage of ray-casting algorithm.

2.4 Determining the Area to See Inside Building

Similar to perception area, we can determine area which if agent stands there it would be able to see inside the building. This is done by exact same ray-casting but with considering building walls.



(a) Perceptible Area of Building (b) Area to see inside

Figure 5: Preceptible Area and Area to see inside of Building

2.5 Fire Simulator

All agents need to predict how fire will spread across the map even if they don't have continuous perception over the fiery buildings, so that they could avoid entering fiery buildings and end up getting burnt out. Also, existence of fire simulator is even more crucial for fire brigades, so they could decide their action better. Our fire simulation is done almost the same way as rescue server's fire simulation. Basically thermal energy transfer between buildings and environment is simulated by two means:

2.5.1 Radiation

Radiation energy transfer coefficient between nearby buildings is calculated by ray-casting algorithm during precomputation phase. The more ray hits, the greater the coefficient would be. Each cycle buildings transfer energy within each other based on their temperature and the coefficient we mentioned before.

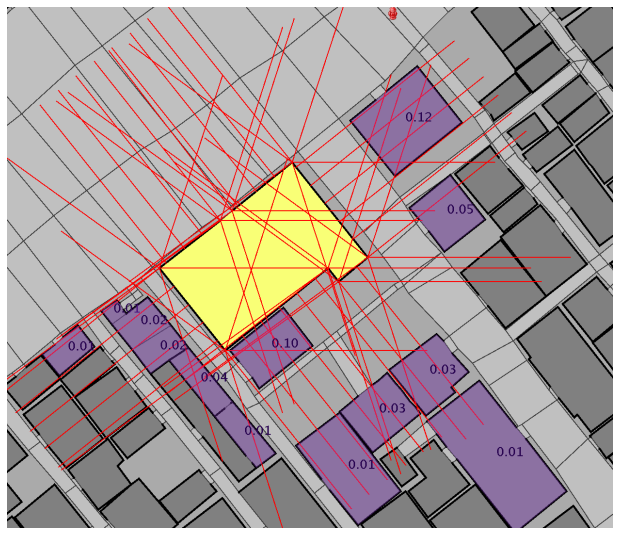


Figure 6: Radiation Thermal Transfer Using Ray-Casting Algorithm

2.5.2 Convection

Convection heat transfer is simulated using 'Air Cells'. Basically air cells are a grid of blocks over the whole map representing the temperature of air in different areas of the map. Each cycle buildings transfer heat with cells directly above them, and also each air cell with its neighbor air cells.

2.6 Viewer (Visual Debugger)

In order to make testing and debugging of each module easier and practical we developed our own viewer module.

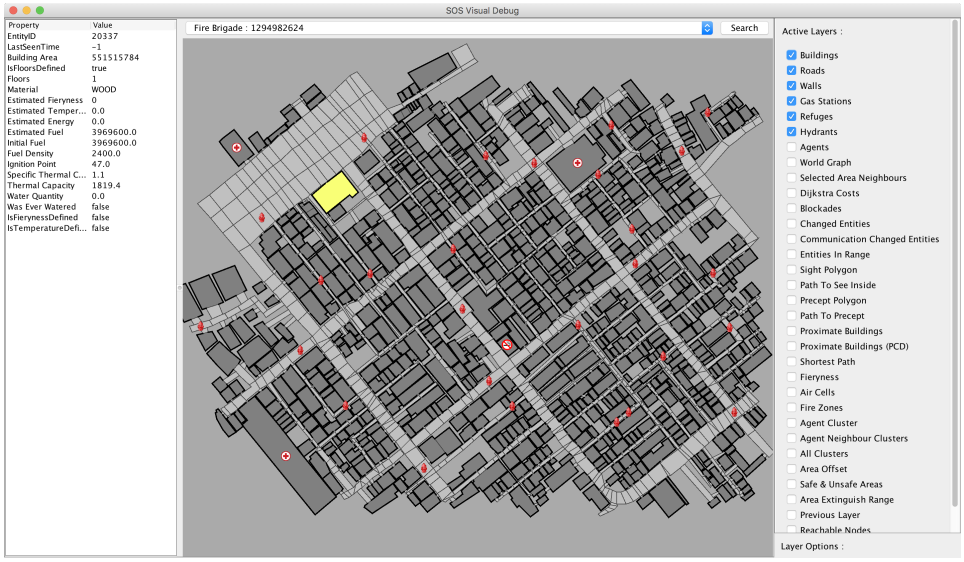


Figure 7: SOS Viewer

3 Sterategy

3.1 Fire Brigade

For better fire extinguishing, after determining fire zones with help of Convex Hull Algorithm we decided to assign agents to each fire zone instead of assigning to clusters. Also we focused on slowing the fire spread instead of fully extinguishing it, because it's sometimes impossible to extinguish the fire completely. here we prioritize buildings at borders of fire zones and also assign another score which is calculated by linear combination of several factors, such as effective radiation of that building, temperature and etc. Coefficient on these factors can be trained using machine learning techniques like genetic algorithms, or neural networks.

3.2 Ambulance

The task of ambulance team is to rescue as many civilians as possible. To achieve this goal ambulance team should first search buildings for humans and then start the rescue process according to some priorities. In search phase each agent first searches their cluster for humans. We define a sight polygon for each building. A sight polygon S for building b is the polygon that if an agent is inside it, it can see the center of building without the need of going inside. In the first phase of our search algorithm we visit cluster buildings by standing in their sight polygon rather than going inside the building. We do so by using a fixed number of nearest buildings to each building, say n. When we have selected a building to search, we sort n nearest buildings to that specific building with respect to their brokenness in reverse order. The next building is selected from the sorted list of buildings (the building with the most brokenness). After visiting each cluster building from its sight polygon each agent selects another cluster with respect to some properties such as distance, communication info, etc. and helps the other agent of that cluster. The goal of this search is to visit all buildings in all clusters from the sight polygon as fast as possible. Then, we use another search to see inside all buildings by actually getting inside those buildings in order to make our search more precise. Sometimes, agents can only move in a limited area of the map. In such conditions agents search buildings which are seen least recently to update information.

For detecting humans to rescue we assign agents to humans. We usually assign one agent to each human, but there are also some cases that two or more agents are assigned to rescue one human. An ambulance agent calculates the time it takes to rescue a damaged human and transport it to refuge, and the time until which the human stays alive. If it can rescue that human before its death time, it starts the rescue process. Because of the errors that might happen in calculation, or wrong prediction due to data imprecision one agent might see another agent rescuing a human knowing that human is not going to survive. In this case, the new agent is also assigned to help improve the speed of rescue process. As mentioned above, we need to predict the time each human has before death. Using heuristic methods, we came up with a formula to predict each damaged human death time upon seeing. We try to rescue humans with the most critical condition to rescue as many civilians as possible. Also at the start damaged agents are prioritized over damaged humans. Then after some time damaged humans are prioritized over agents.

3.3 Police Force

Police force is the key agent. If Police doesn't work well, all other agents work would be greatly affected. Therefore learning from our experience in Asia pacific competition, we have put more focus on this agent. Police main job is to clear blockades off the areas. Areas are scored based on their practicality (for instance Refuges, Hydrants, normal buildings, ...), distance to agent, whether they contain civilians or not, brokenness, and their reachability to extinguish range of fiery buildings. In the beginning each Police force searches and chooses its target inside their assigned cluster. when a target is chosen, they take action to move to the target location. On their way to the target if their road is blocked, they clear off the blockade. They don't necessarily clear all the blockade. It's cleared in an efficient way so that an agent can move without getting stuck. They also check for stuck agents and civilians on their path and release them. Buildings with blocked entrances are cleared too. After visiting all their cluster targets, they choose another cluster based on a given score to help another Police force. In the end if all the clusters are cleared, Police forces scouts the map to help other agents (Ambulance team and Fire brigade).

4 Results

Results of running simulation on different maps are given in the following table :

Team	Map			
	Eindhoven3	Istanbul2	Paris2	VC3
SOS-CS	93.1	20.6	35.5	132.2
Aura	115.7	32.9	46.9	170.3
MRL	82.2	25.3	43.7	145.6
RoboAKUT	93.6	12.7	37.5	142.9

5 References

1. RoboCup Rescue 2014 – Rescue Simulation League Infrastructure Paper S.O.S (Iran)
2. RoboCup Rescue 2015 – TDP Rescue Simulation S.O.S (Iran)
3. KMeans++: The Advantages of Careful Seeding - David Arthur and Sergei Vassilvitskii
4. A density-based algorithm for discovering clusters in large spatial databases with noise - Martin Ester